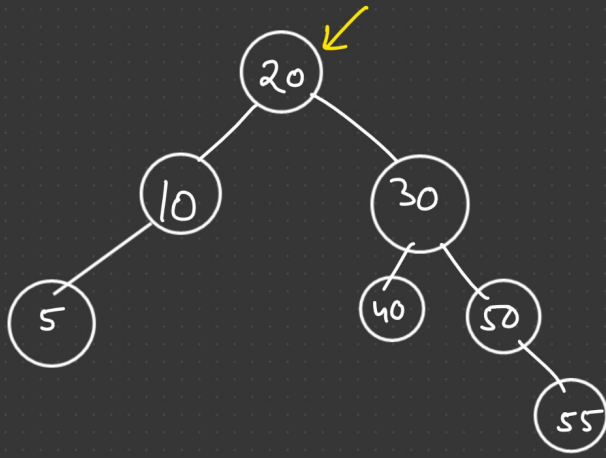


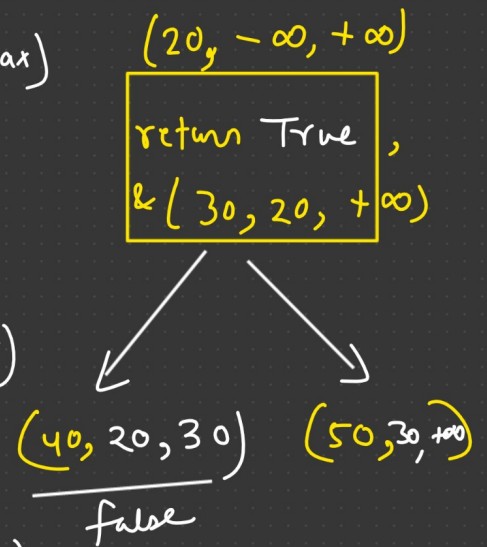
Validate BST:-



```
bool isValidBST (node* root, int min, int max)
{
    if (root == NULL)
        return true;

    if (root->data < min || root->data > max)
        return false;

    return (isValidBST (root->left, min, root->value)
        && isValidBST (root->right, root->value, max));
}
```



Find min & max value of BST:-

↓
left most

↓
Right most

```
node * min( node * root)
{
    node temp = root;

    while( temp->left != null)
    {
        temp = temp->left;
    }

    return temp;
}
```

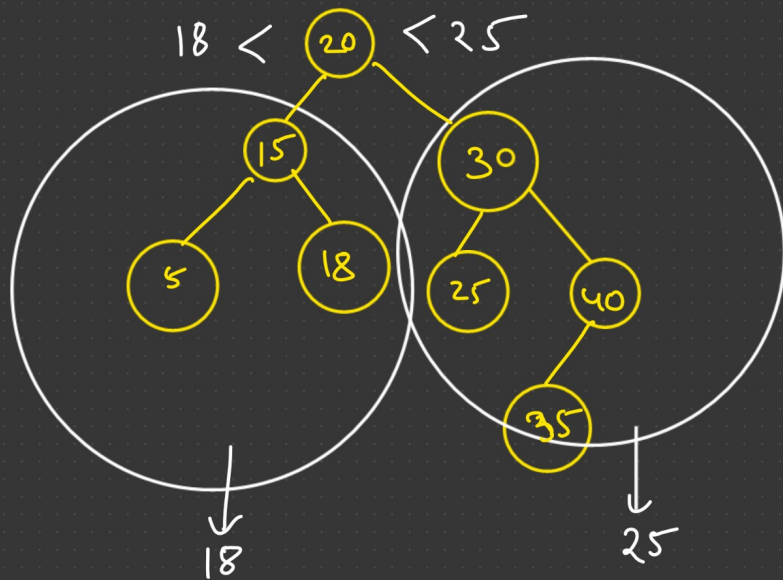
```
node * max( node * root)
{
    node temp = root;

    while( temp->right != null)
    {
        temp = temp->right;
    }

    return temp;
}
```

Inorder Successor & Inorder Predecessor of BST:-

*// BST $\rightarrow$ inorder Traversal $\Rightarrow$ Sorted sequence



5, 15, 18, 20, 25, 30, 35, 40

pred = max of left subTree

Succ = min of right subTree

```
node* inorderSucc(node* root)
{
    return min(root->right);
}
```

```
node* inorderPred(node* root)
{
    return max(root->left);
}
```


Insertion/Creation of BST:-

```
node * insert (node * root, int value)
{
```

```
    if (root == null)
```

```
        return new node(value);
```

```
    if (root->data < value)
```

```
        root->right = insert(root->right, value);
```

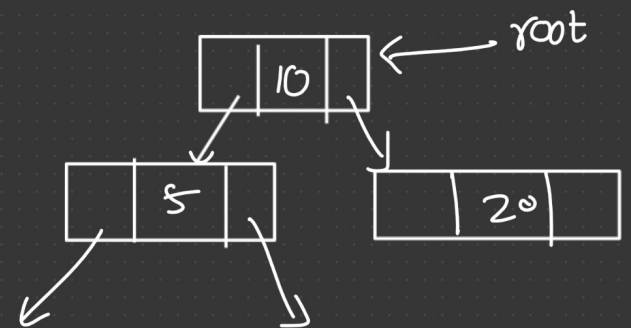
new node(value);

```
    if (root->data >= value)
```

```
        root->left = insert(root->left, value);
```

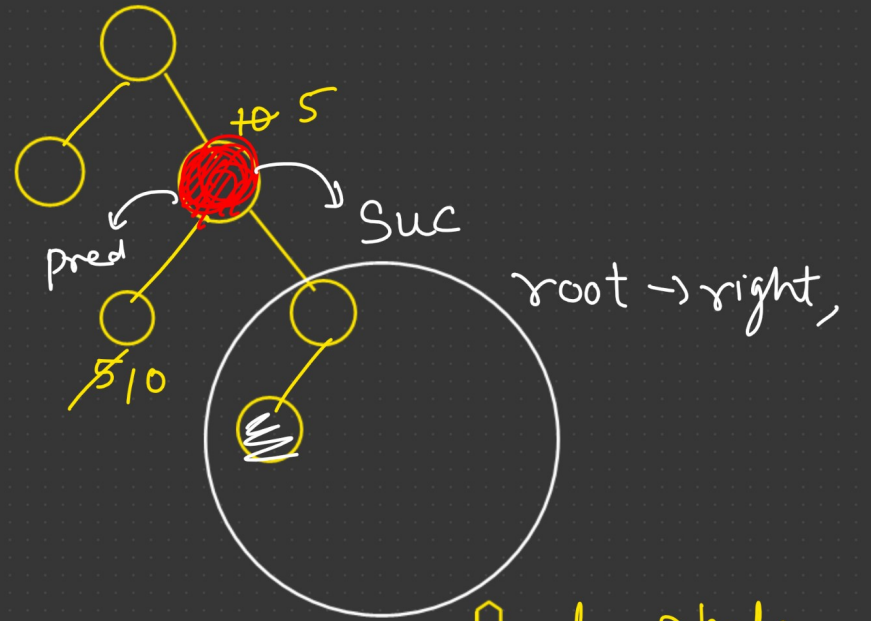
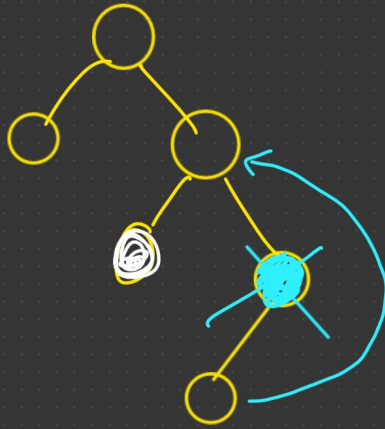
new node(value)

```
}
```



Deletion of BST :-

- 1) No child node 
- 2) One child node 
- 3) Two child node 



Pred = phole
Suc = Baad me

```
node * deletion (node * root, int value)
```

```
{
```

```
    if (root == null)
```

```
        return root;
```

```
    if (root → data < value)
```

```
        root → right = deletion (root → right, value);
```

temp

```
    else if (root → data > value)
```

```
        root → left = deletion (root → left, value);
```

temp

```
    else {
```

// Zero child

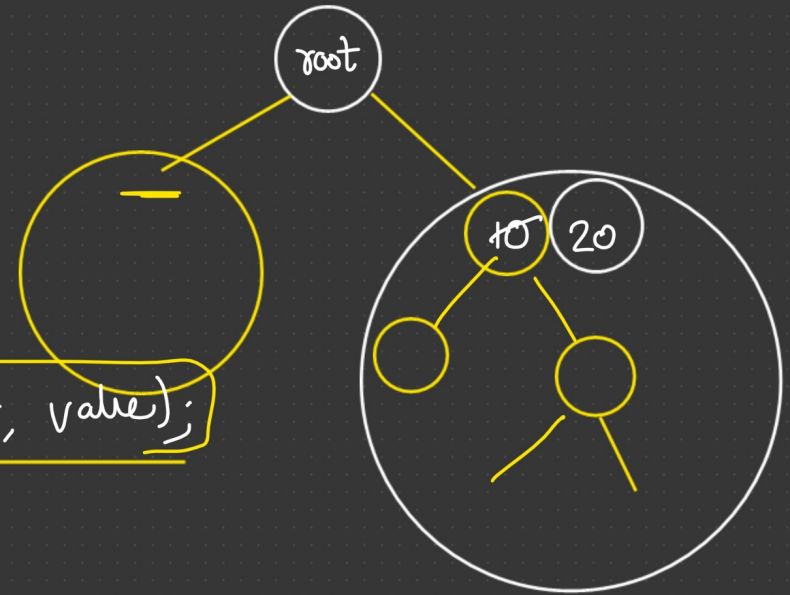
```
        if (root → left == Null && root → right == Null)
```

```
        {
```

```
            delete root;
```

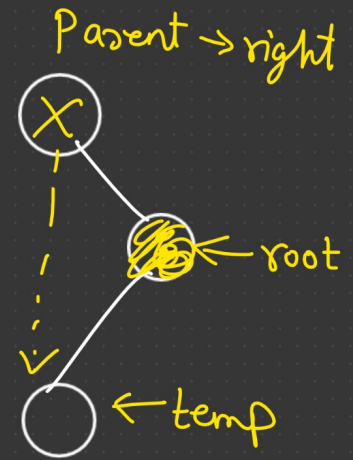
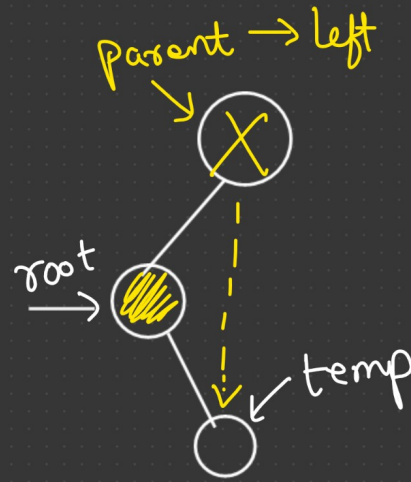
```
            return null;
```

```
        }
```



// one child

```
else if ( root → left == Null )  
{  
    node *temp = root → right;  
    delete root;  
    return temp;  
}
```

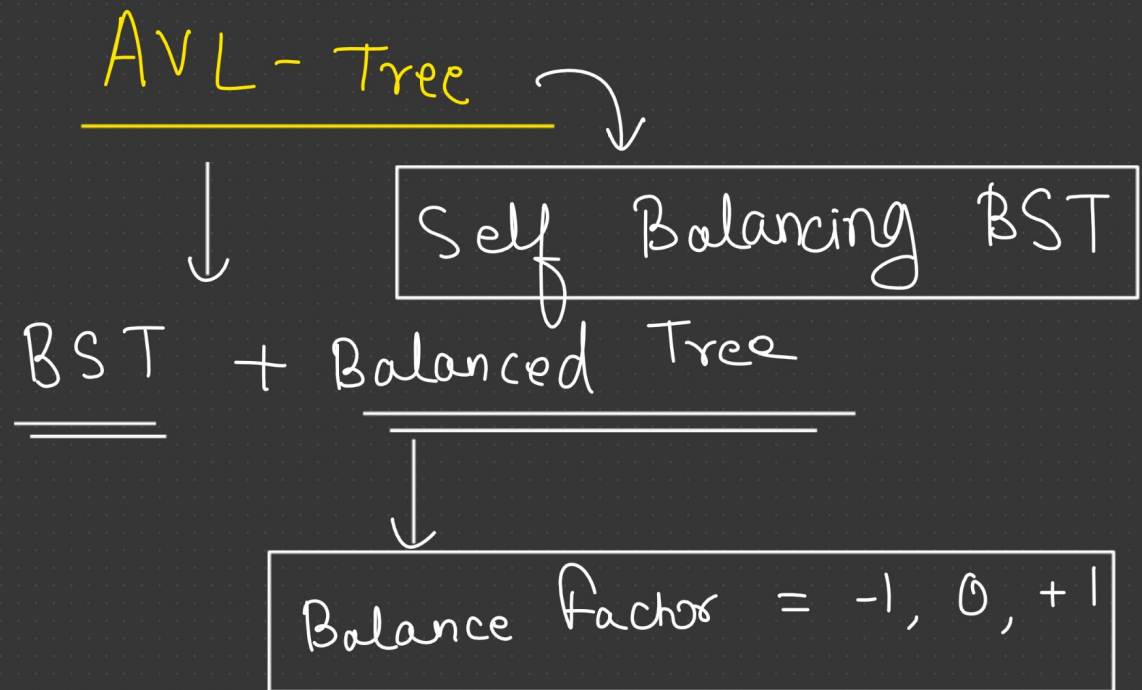


```
else if ( root → right == null )  
{  
    node *temp = root → left;  
    delete root;  
    return temp;  
}
```

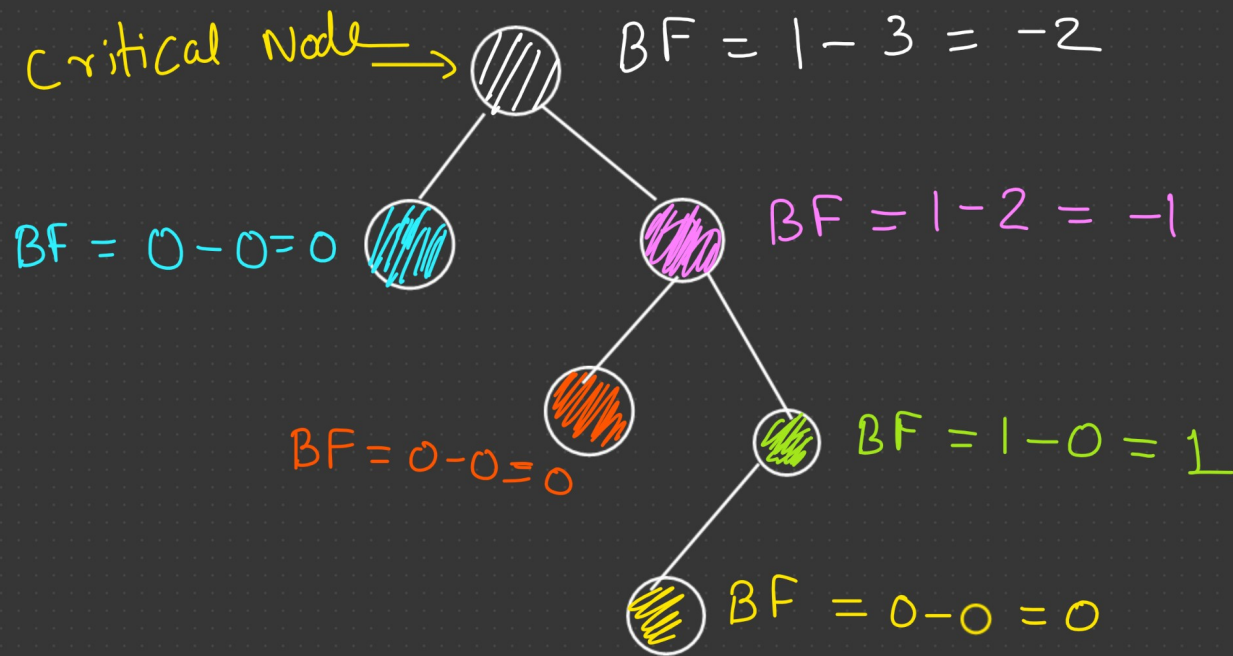
// Two child

```
else {  
    node *temp = inorderSuccessor(root);  
    root → data = temp → data;  
    root → right = deletion( root → right, temp → data );  
}
```

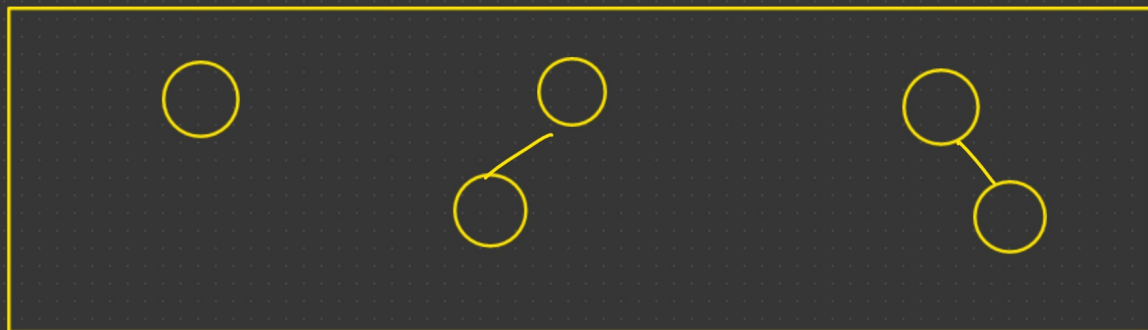
```
}  
return root;  
}
```



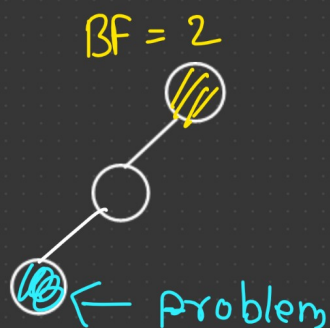
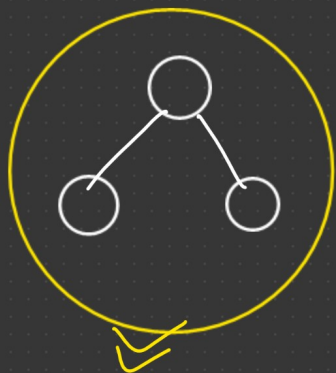
Balance factor = left height - right height



This is not
a Balanced Tree

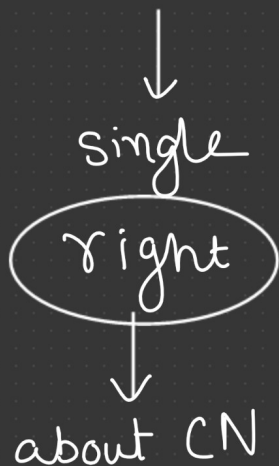


→ Always Balance



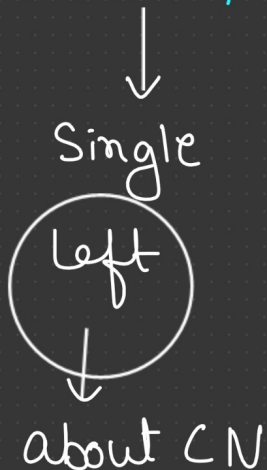
Left → Left

① LL Rotation



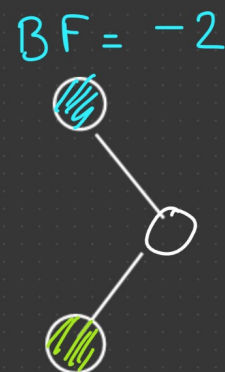
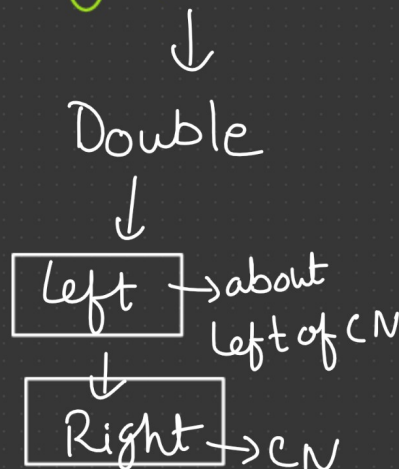
Right → Right

② RR Rotation



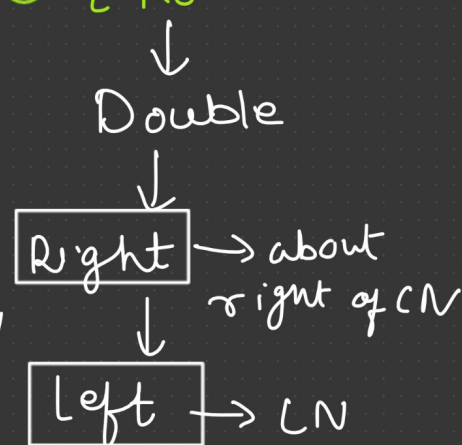
Left → Right

③ LR Rotation

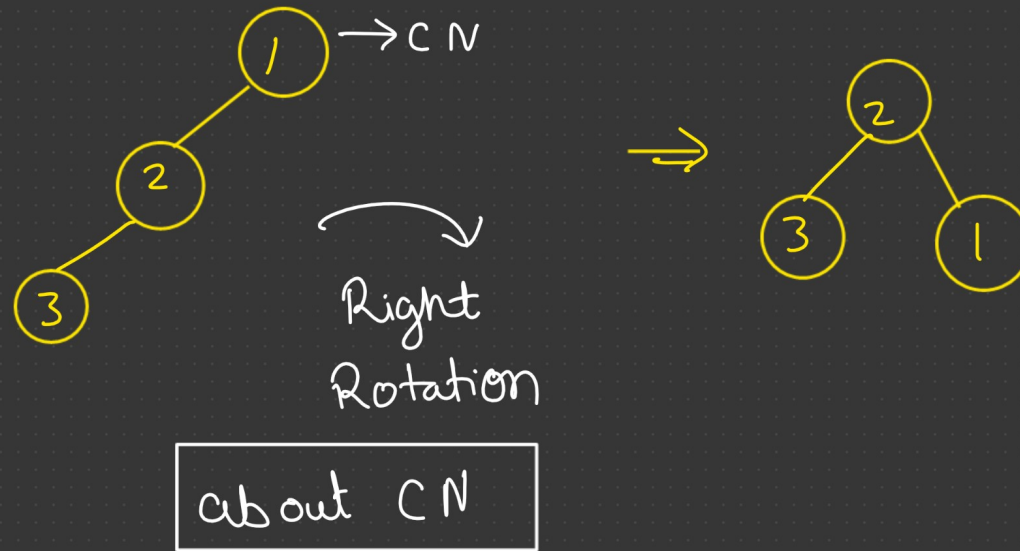


Right → Left

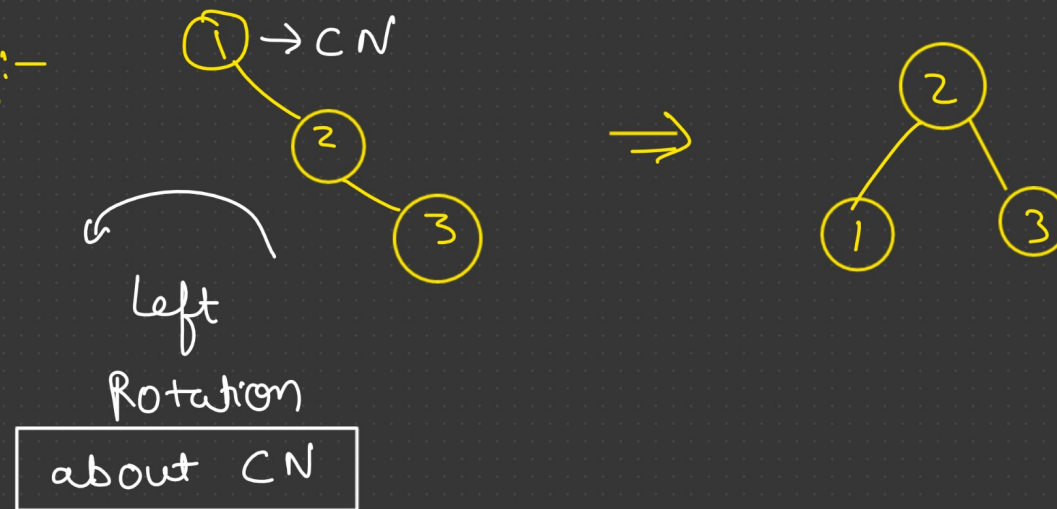
④ RL Rotation



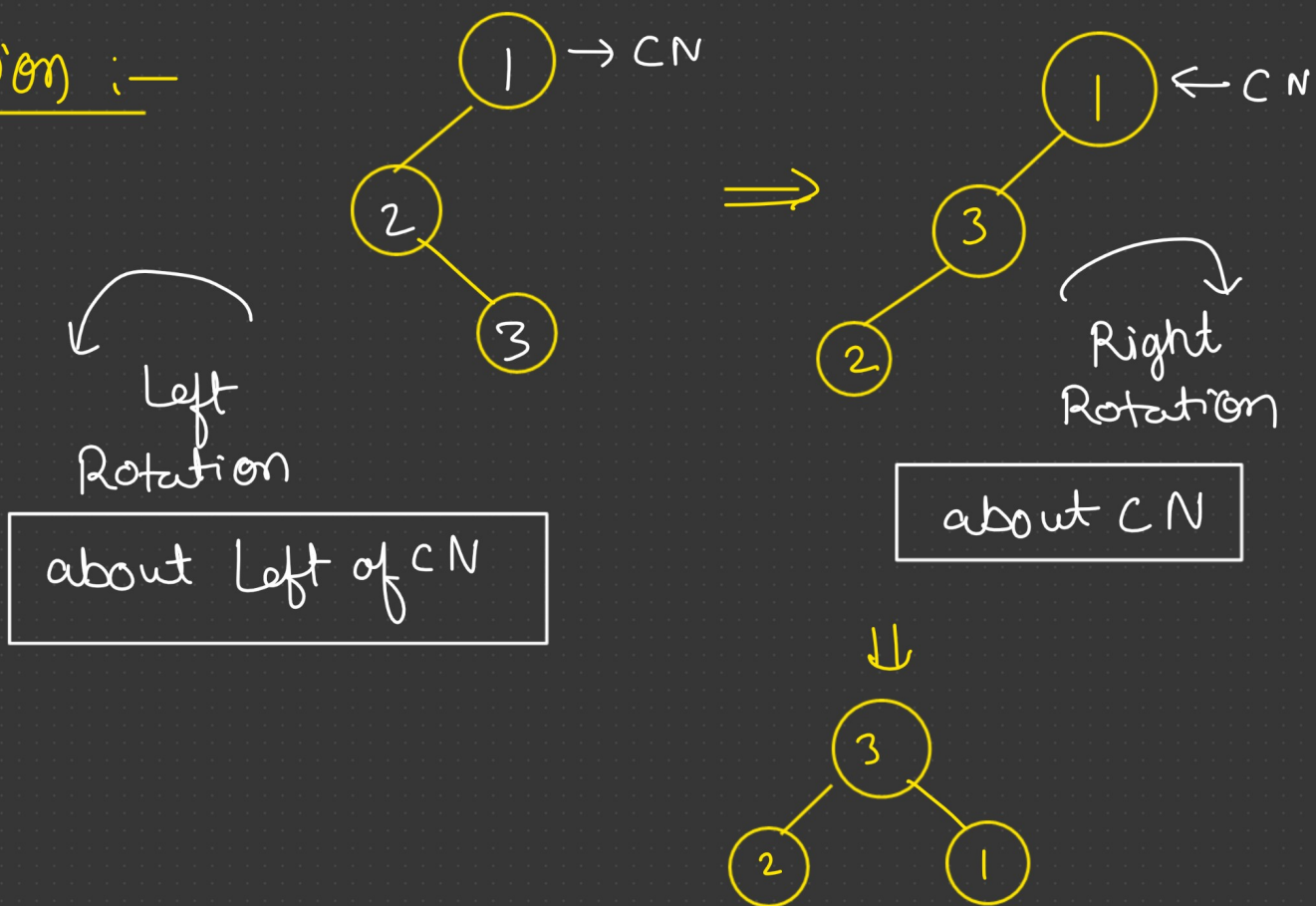
① LL Rotation :-



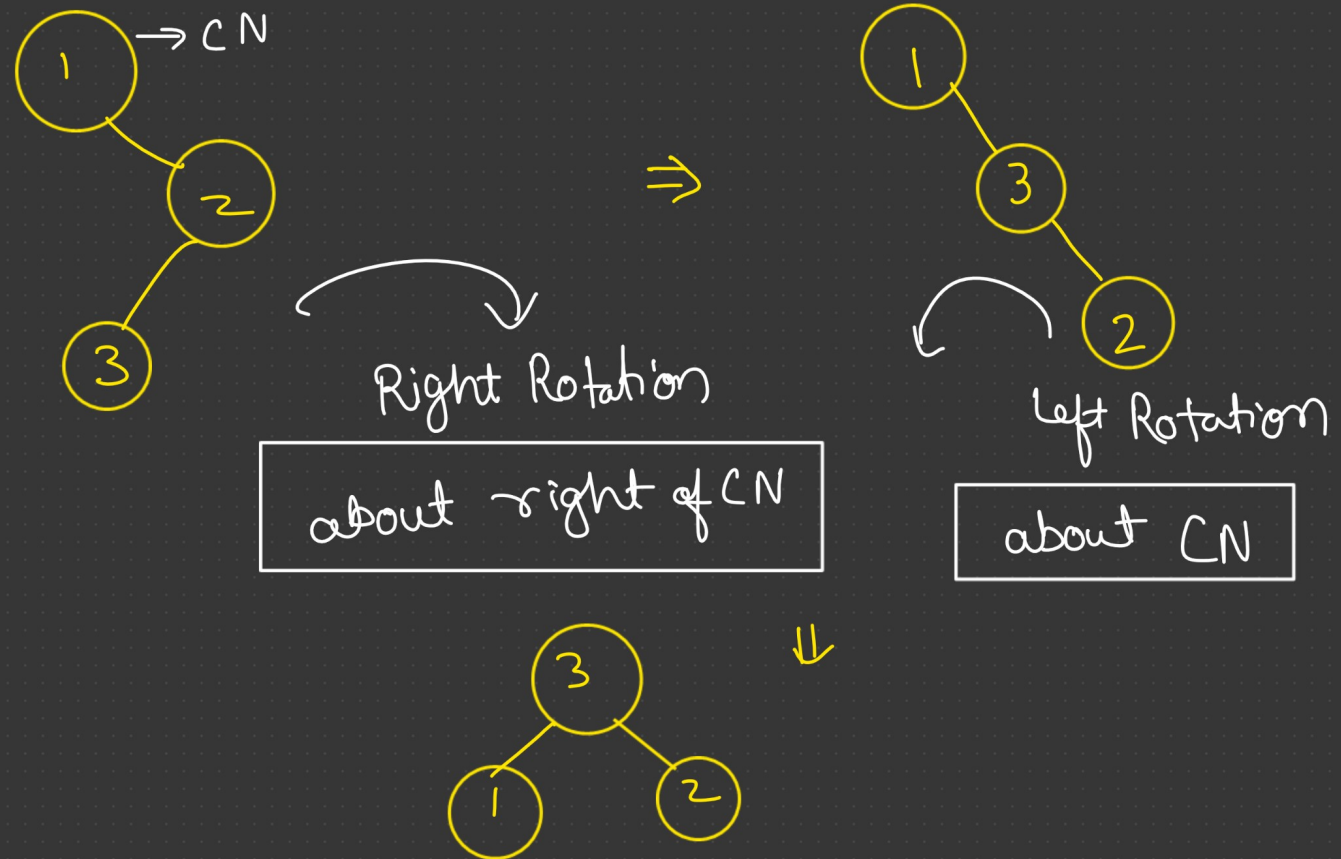
② RR Rotation :-



③ LR Rotation :-



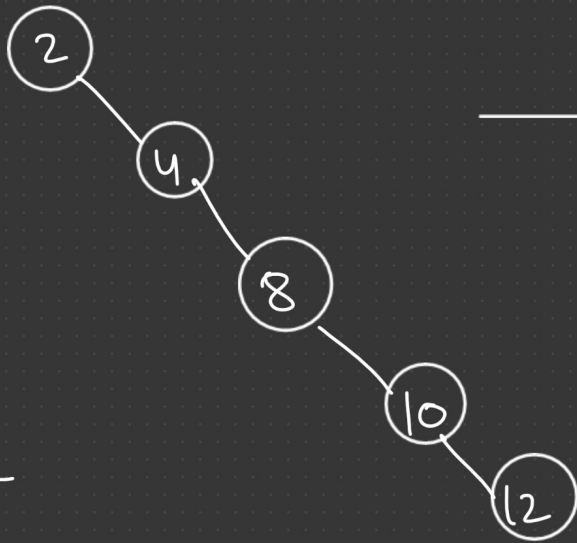
④ RL Rotation :-



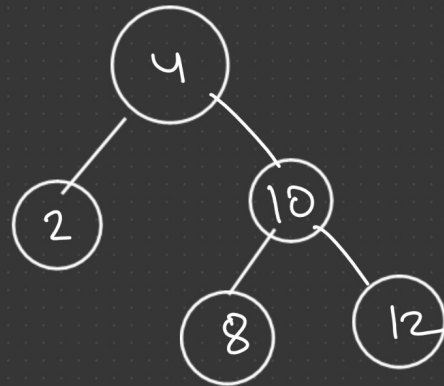
Benefit of AVL Tree :-

~~2~~, ~~4~~, ~~8~~, ~~10~~, 12

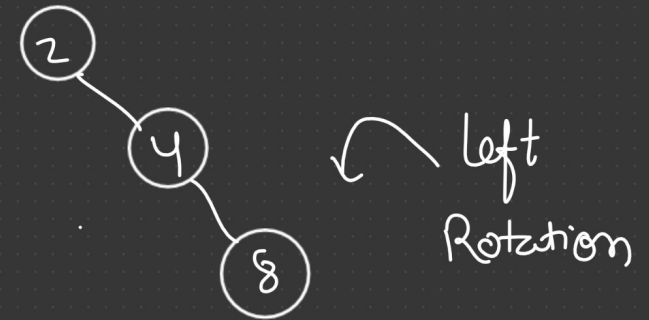
$\rightarrow O(n)$



Skew Tree

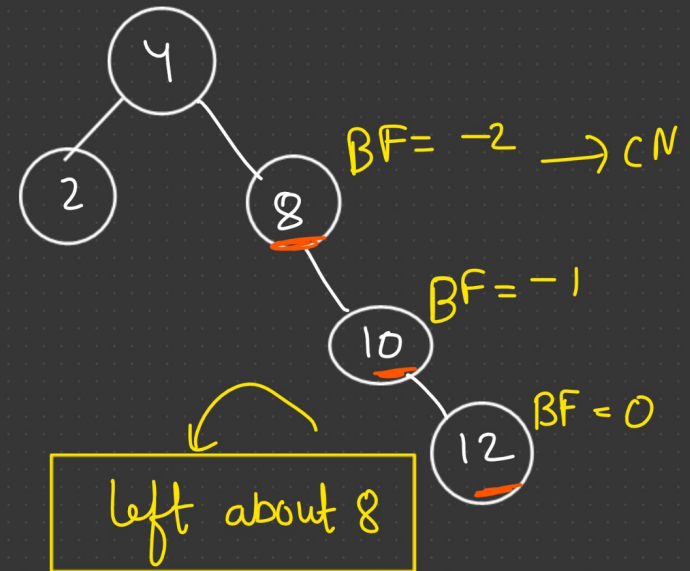


$O(\log n)$

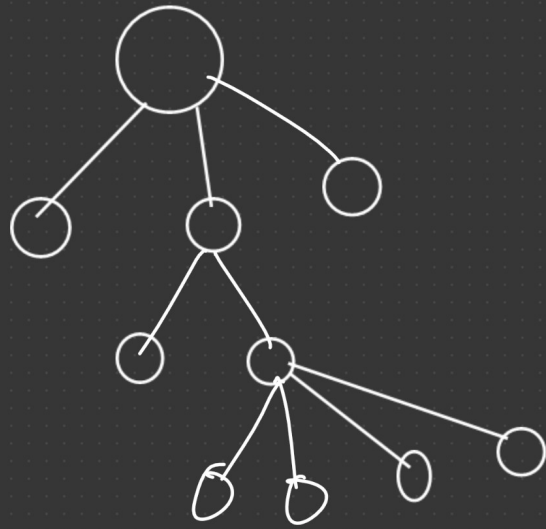


Left Rotation

RR Rotation

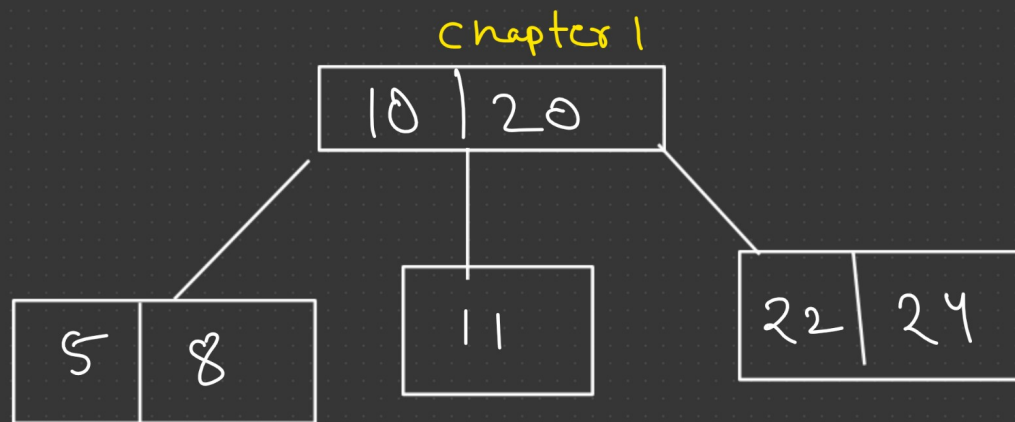


General Tree



n-ary Tree

M-way Tree / Multiway Tree



m = order of the tree

$$\log n = \text{height of tree}$$

(BST) + Balance

2-way tree (order = 2)

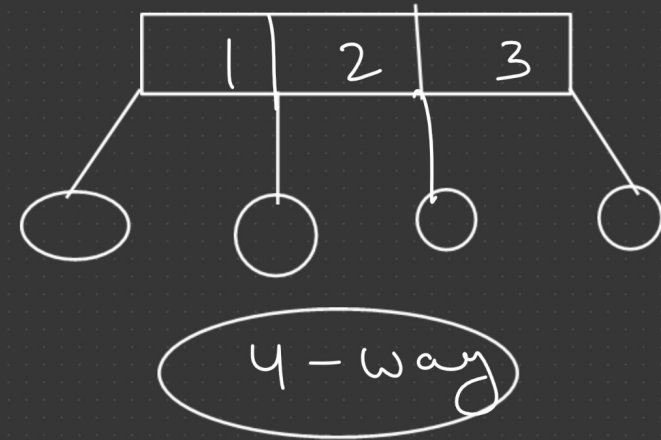
3-way tree (order = 3)

4-way tree (order = 4)

m-way Tree

$m = \text{order} = \text{max. no. of child}$

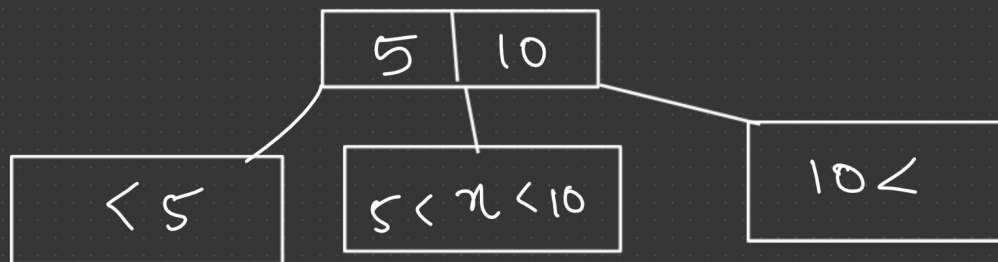
$m-1 = \text{max. no. of elements in a node}$



3-way Tree:-

3 = max child

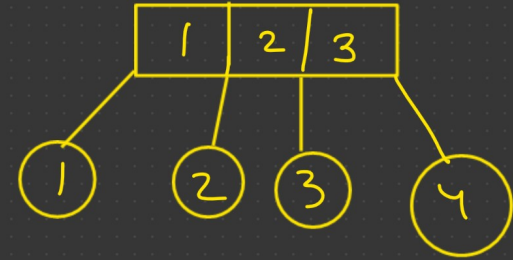
2 = max element each node



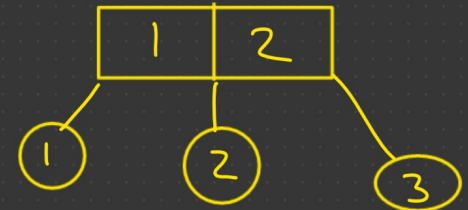
2-way = BST

B-Tree

✶ It is an m-way Tree



✶ All leaf nodes are at same level



✶ $m = \text{max child}$
 $\lceil \frac{m}{2} \rceil = \text{min child}$

$m-1 = \text{no. of max element}$
 $\lceil \frac{m}{2} \rceil - 1 = \text{no. of min element}$

↓
↓
except root node

Create B-Tree of order 4. having elements

10, 5, 4, 20, 15, 25, 30, 32, 45

$$m = 4$$

→ max 4 child

$$m-1 = 3$$

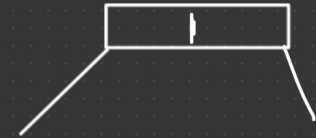
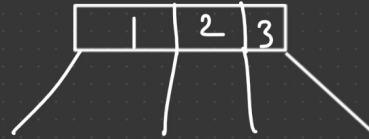
→ max 3 element

$$\frac{m}{2} = \frac{4}{2} = 2$$

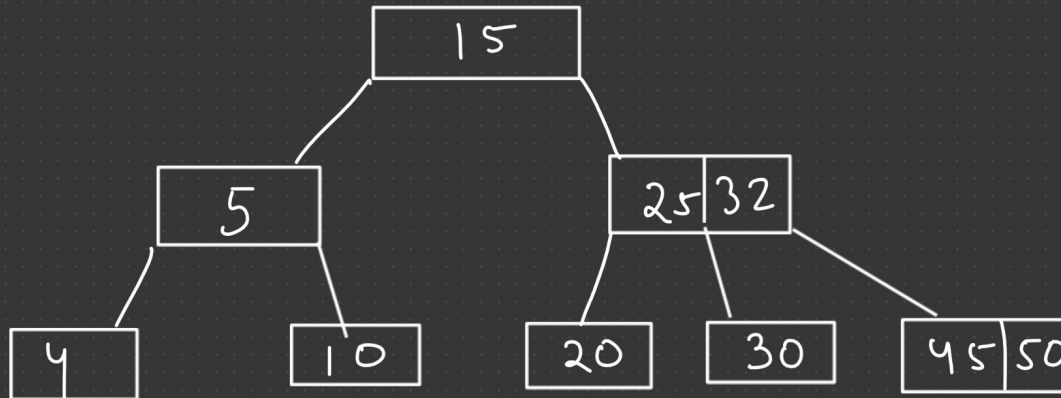
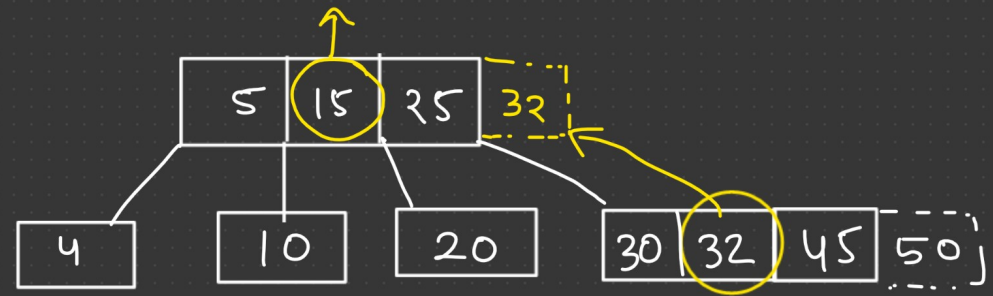
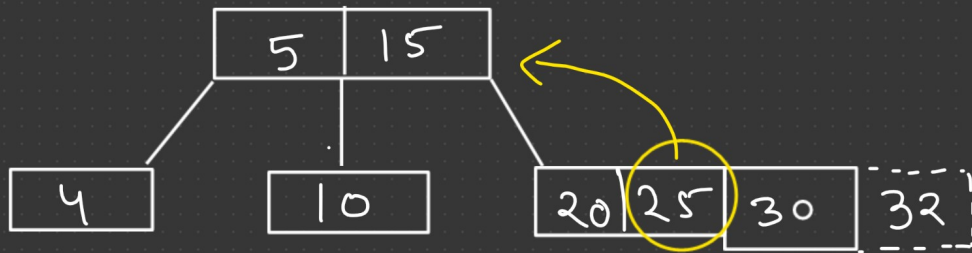
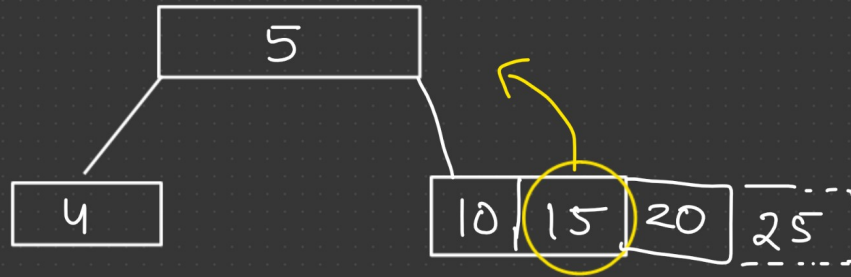
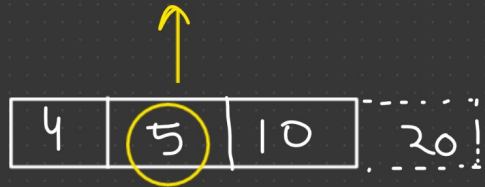
→ min 2 child

$$\frac{m}{2} - 1 = 2 - 1 = 1$$

→ min 1 element



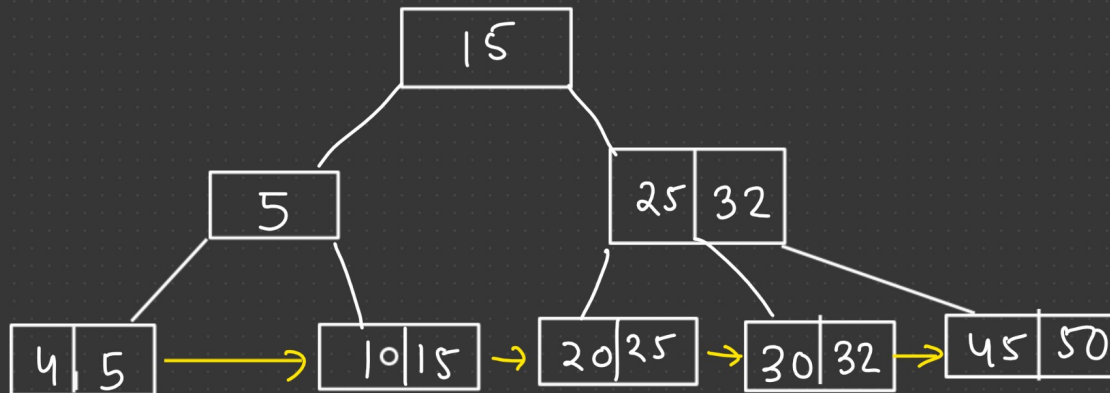
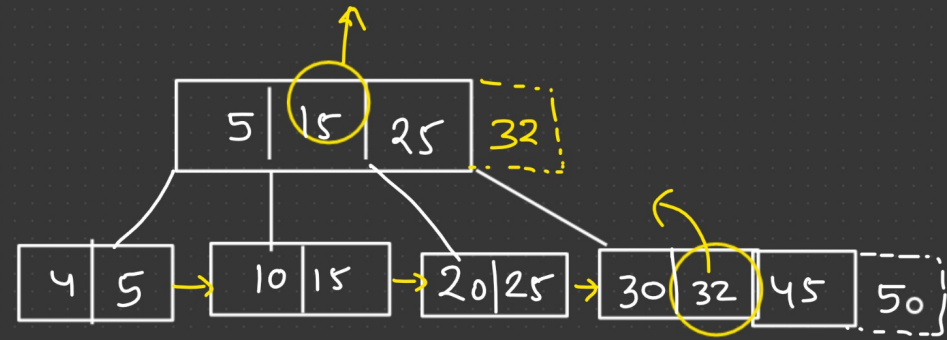
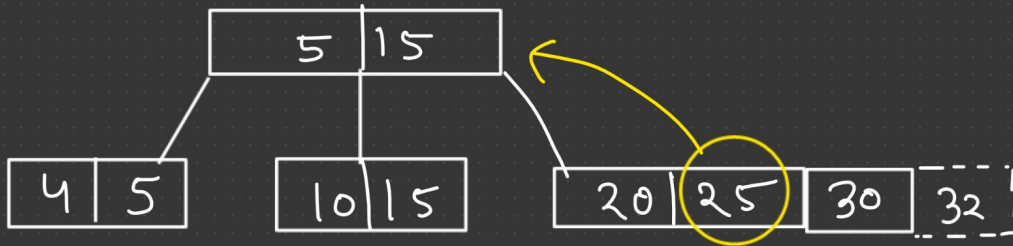
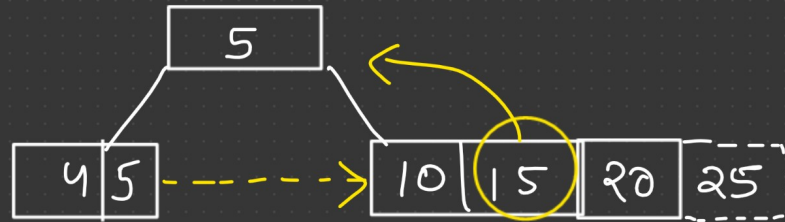
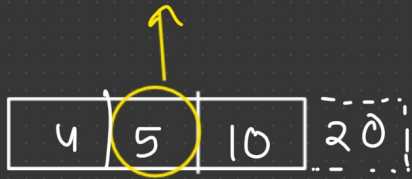
~~10, 8, 4, 20, 15, 25, 30, 32, 45, 50~~



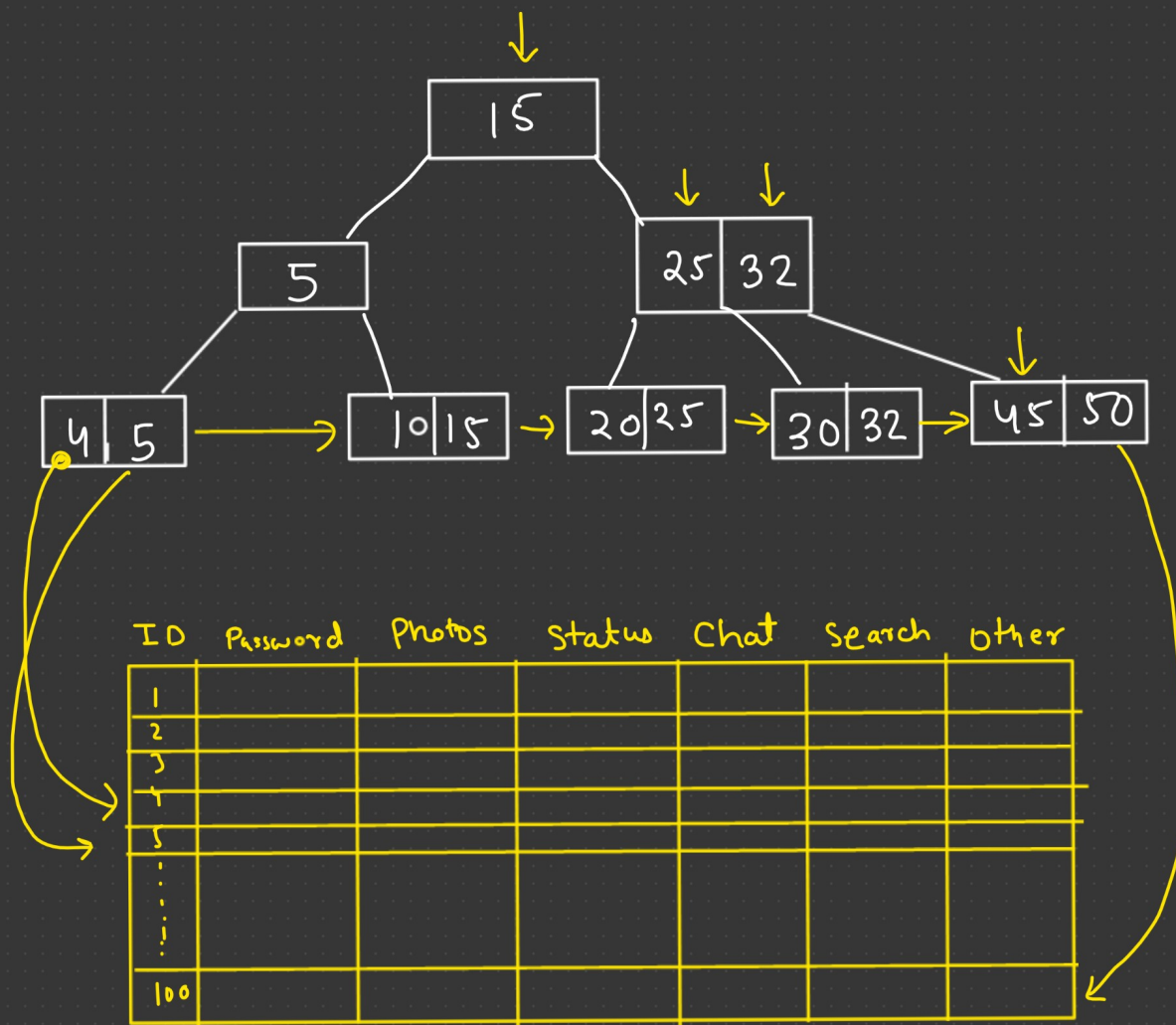
B+ Tree

$m = 4$

~~10, 8, 4, 20, 15, 25, 30, 32, 45, 50~~



Indexing



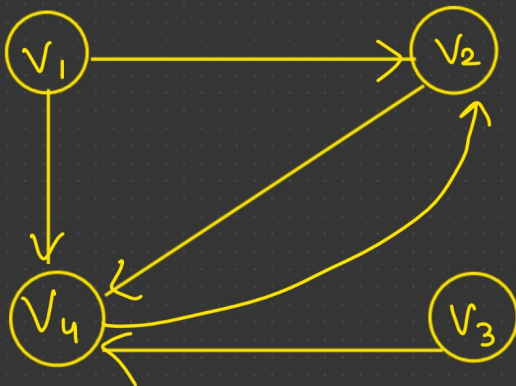
Graph

A graph is defined as an ordered set $G(V, E)$.

where V is vertex, E is edge

$$V = \{v_1, v_2, v_3, v_4\} \rightarrow \text{vertex}$$

$$E = \left\{ \underbrace{(v_1, v_2)}_{e_1}, \underbrace{(v_2, v_4)}_{e_2}, \underbrace{(v_1, v_4)}_{e_3}, \underbrace{(v_3, v_4)}_{e_4}, \underbrace{(v_4, v_2)}_{e_5} \right\} \rightarrow \text{edge}$$



Directed Graph

Graph



1) Undirected
Graph



2) Directed
Graph

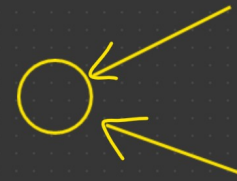


3) Weighted
Graph

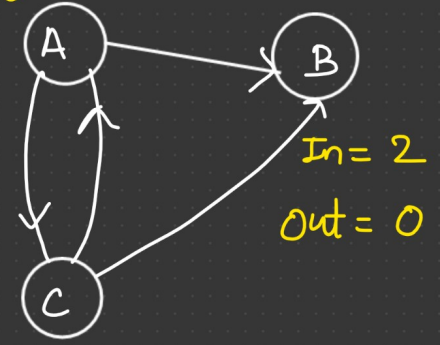


Degree of nodes

- 1) Indegree
- 2) Outdegree



Indegree = 1
outdegree = 2



Indegree = 1
outdegree = 2

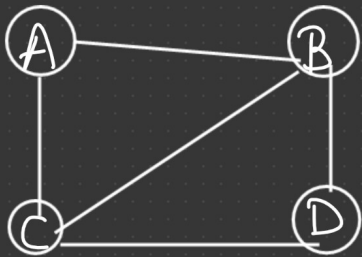
For undirected graph :- $\text{Indegree} = \text{Outdegree}$

Graph Representation

1) Adjacency Matrix

2) List Representation

i) Adjacency Matrix:- (when graph is dense)



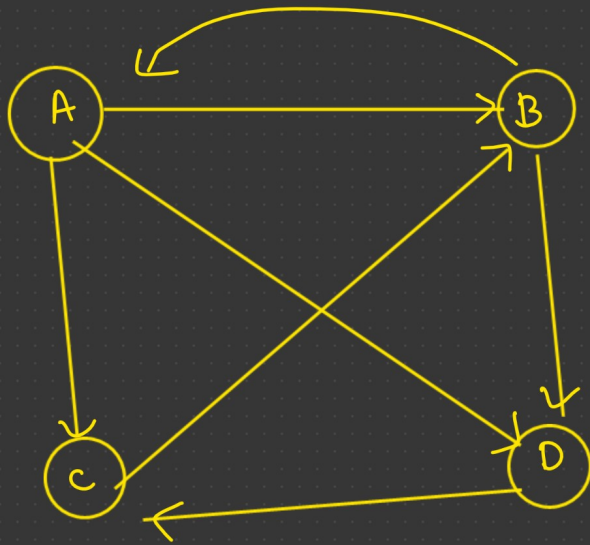
	A	B	C	D
A	1	1	1	0
B	1	1	1	1
C	1	1	1	1
D	0	1	1	1

4x4

$$A^T = A$$

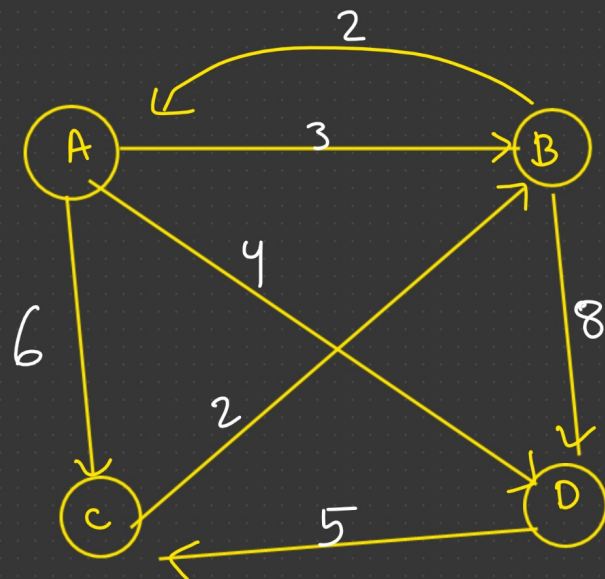
$$a[1][2] = 1$$
$$a[2][1] = 1$$

→ for undirected graph (Symmetric Matrix)



$q[\text{Row}][\text{col}]$

	A	B	C	D
A	1	1	1	1
B	1	1	0	1
C	0	1	1	0
D	0	0	1	1



	A	B	C	D
A	0	3	6	4
B	2	0	0	8
C	0	2	0	0
D	0	0	5	0

```
int a[4][4];
```

```
for (i=0 ; i<4 ; i++) —→ start / source
```

```
{
```

```
    for (j=0 ; j<4 ; j++) —→ stop / destination
```

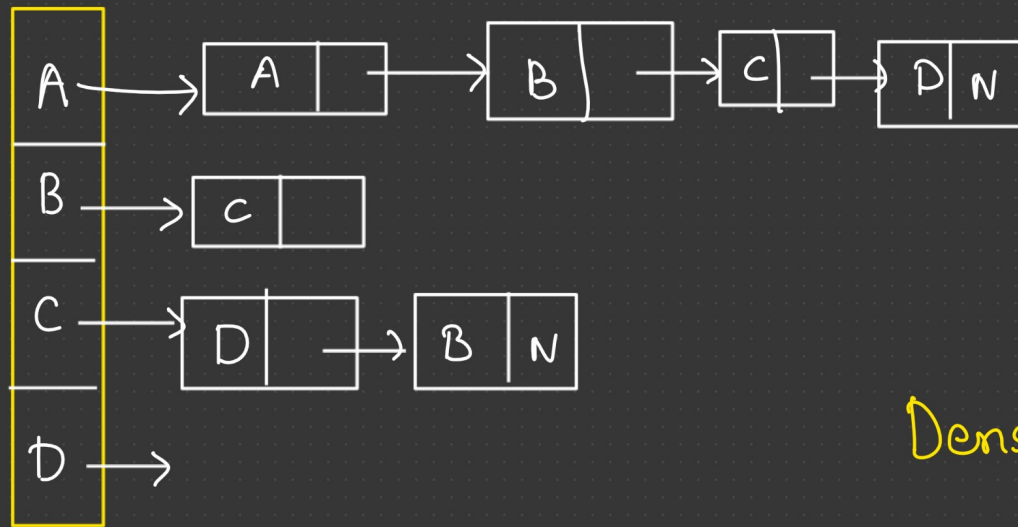
```
    {
```

```
        cin >> a[i][j];
```

```
    }
```

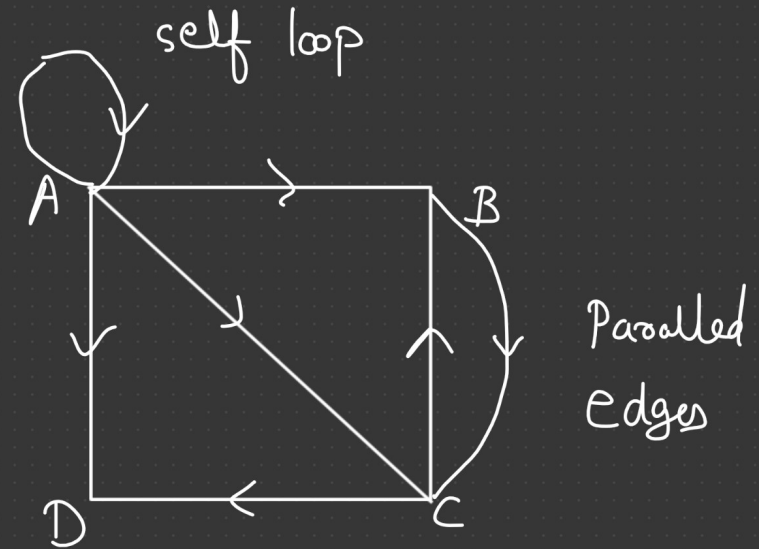
```
}
```


2) Adjacency List :- (used when graph is sparse)



Array of pointers

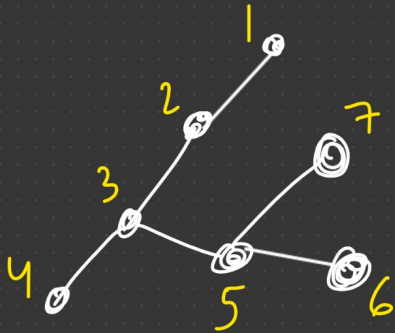
Dense ↑
sparse ↓



Traversal

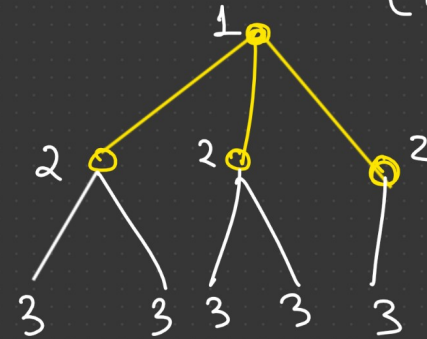
DFS (Depth First Search)

(Preorder)



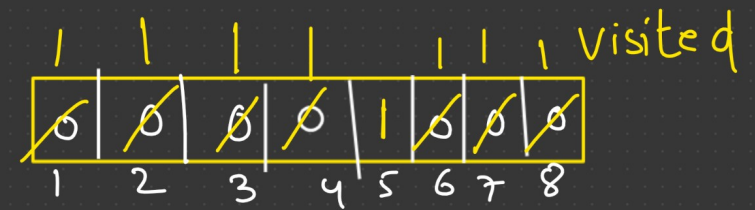
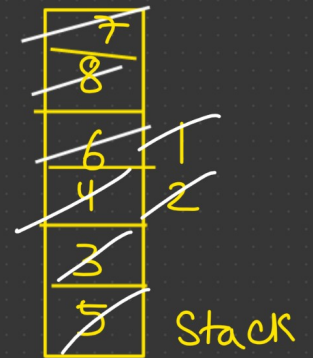
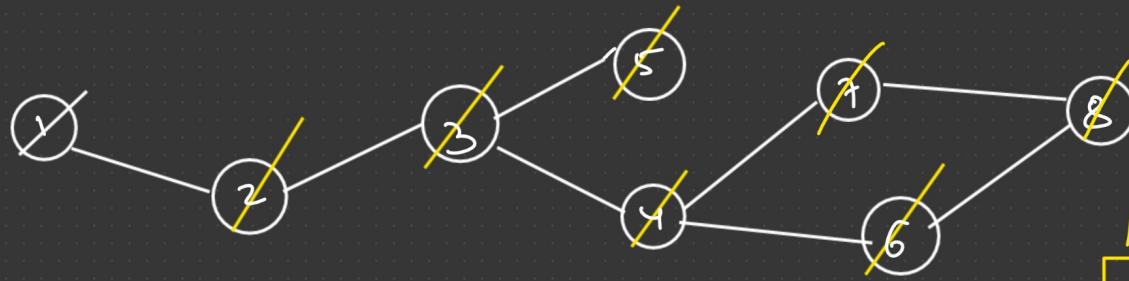
Backtracking
(Stack)

BFS (Breadth First Search)
(Level)



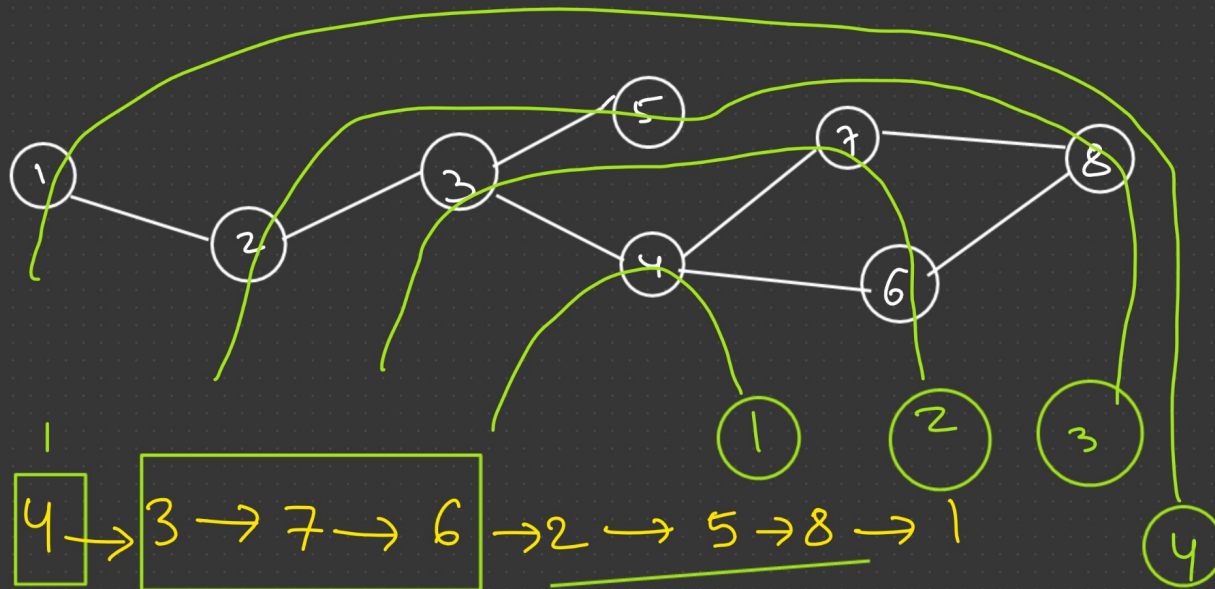
Queue

DFS (Depth First Search)



5 → 3 → 4 → 6 → 8 → 7 → 2 → 1

BFS (Breadth First Search)



4 → 3 → 7 → 6 → 2 → 5 → 8 → 1

2

